

Practical 6

Aim :

Prepare System Analysis and System Design of identified Requirement specifications using structure design as DFD with data dictionary and Structure chart for the specific module.

Objectives :

- Understand the requirements of an Online Trading System.
- Perform system analysis to identify functional and non-functional requirements.
- Design the system using structured design techniques:
 - Create Data Flow Diagrams (DFD).
 - Develop a Data Dictionary.
 - Design a Structure Chart for the system module.
- Document the system design for implementation.

Prerequisites :

- Basic understanding of system analysis and design.
- Familiarity with DFD, Data Dictionary, and Structure Chart.
- Knowledge of database concepts.
- Tools: Draw.io, Lucidchart, Visio, or pen & paper.

Lab Setup :

- Software: Diagramming tool
- Hardware: Computer with internet
- Reference Material: Requirement specification of Online Trading System

Task 1: System Analysis

Requirement Gathering :

Key Stakeholders :

- Trader / Investor
- Admin
- Bank Payment Gateway
- Stock Exchange (Simulated)

Functional Requirements :

- User Registration
- Login/Logout
- View Stock Prices
- Buy Shares
- Sell Shares
- View Portfolio
- View Transaction History
- Admin can add/update/remove stocks
- Admin can monitor transactions

Non-Functional Requirements :

- Secure Authentication
- Fast transaction processing
- Data integrity
- High availability
- User-friendly interface

Requirement Specification

Input:

- User details
- Login credentials
- Stock order details (Buy/Sell)
- Payment detail

Process:

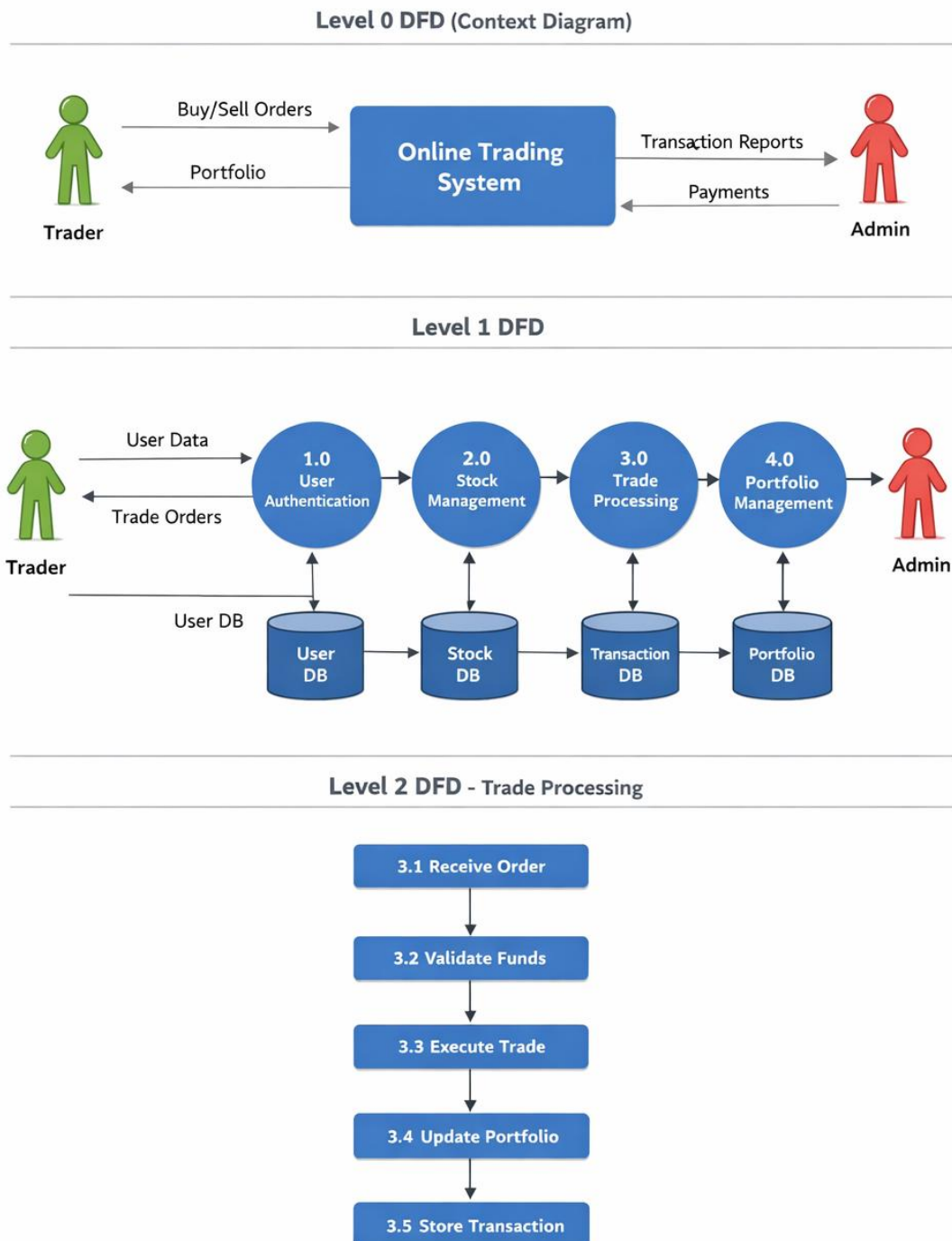
- Validate user
- Check stock availability
- Process transaction
- Update portfolio
- Store transaction record

Output:

- Transaction confirmation
- Updated portfolio
- Reports
- Admin monitoring dashboard

Task 2: System Design Using Structured Design

1. Data flow diagram



2. Data Dictionary

Data fields :

Data Element	Description	Data Type	Example
User_ID	Unique user identifier	String	U1001
User_Name	Full name	String	Arsh Patel
Email	User email	String	arsh@gmail.com
Password	Encrypted password	String	*****
Stock_ID	Unique stock code	String	STK001
Stock_Name	Name of stock	String	TCS
Stock_Price	Current stock price	Float	3500.50
Quantity	Number of shares	Integer	10
Transaction_ID	Unique transaction ID	String	TXN1001
Transaction_Type	Buy/Sell	String	Buy
Transaction_Date	Date of trade	DateTime	2026-02-25
Portfolio_Value	Total investment value	Float	50000

Data Stores :

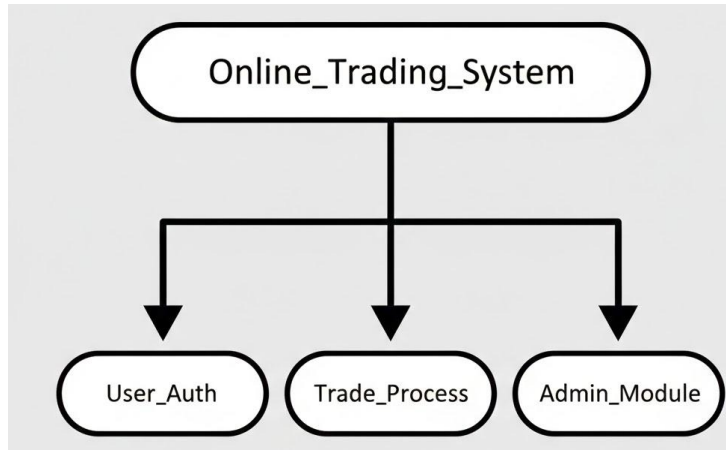
Data Store	Description	Key Fields
User_DB	Stores user details	User_ID
Stock_DB	Stores stock information	Stock_ID
Portfolio_DB	Stores user holdings	Stock_ID
Transaction_DB	Stores trade history	Transaction_ID

Data flow:

Source → Destination	Description
1 User = System	
2 User = System	Authentication request
2 User = System	Buy/Sell order
3 System = User	Confirmation message
4 Admin = System	Update stock price
4 Admin = System	

3. Strucuture Chart :

Level 0 (Main Module) :



Level 1 : Breakdown:

1. User Auth:

- validate credentials
- assign user role

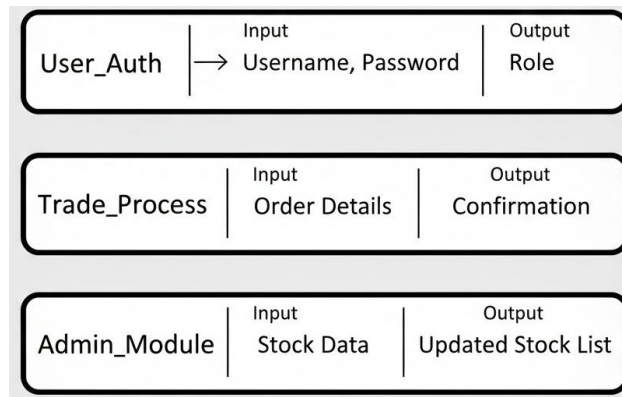
2. Trade Process :

- receive order
- check balance
- execute trade
- update portfolio

3. Admin Module :

- add stock
- update stock
- remove stock
- view reports

Key data flow in structure chart:



Task 3: Documentation

Include in Final Report:

- Requirement Specification
- DFD Level 0, 1, 2
- Data Dictionary
- Structure Chart
- Explanation of each diagram

Expected Output

- ✓ System Analysis Document
- ✓ DFD Diagrams
- ✓ Data Dictionary
- ✓ Structure Chart
- ✓ Summary of Design Decisions

Practical 7

Aim :

Designing the module using Object Oriented approach including a Use case Diagram with scenarios, Class Diagram and State Diagram, Collaboration Diagram, Sequence Diagram, and Activity Diagram.

Objectives :

- Understand the requirements of an Online Trading System.
- Apply Object-Oriented Programming (OOP) principles to design the system.
- Create UML diagrams to represent system structure and behavior.
- Model trading operations such as buying, selling, and portfolio management.

Prerequisites :

- Basic knowledge of Object-Oriented Programming concepts.
- Familiarity with UML diagrams.
- Understanding of database and web applications.
- Tools: Draw.io, StarUML, Lucidchart, or any UML tool.

Lab Setup :

- Software: UML Diagram Tool, IDE (Java/Python)
- Hardware: Computer with internet access
- References: Online Trading platforms and UML documentation

Task 1: Requirements Analysis

Objective :

- Identify functional and non-functional requirements of the Online Trading System.

System Purpose :

- The Online Trading System allows users to buy and sell stocks electronically, manage portfolios, and track transactions securely.

Stakeholders :

- Trader (User)
- Admin
- Bank Payment System
- Stock Market (Simulated)

Functional Requirements

- User registration and login
- View stock prices
- Buy stocks
- Sell stocks
- View portfolio
- Transaction history
- Admin stock management
- Generate reports

Non-Functional Requirements

- Secure authentication
- Fast transaction processing

- Data consistency
- High availability
- User-friendly interface

Task 2: Use Case Diagram with Scenarios

Actors :

- Trader
- Admin
- Payment Gateway

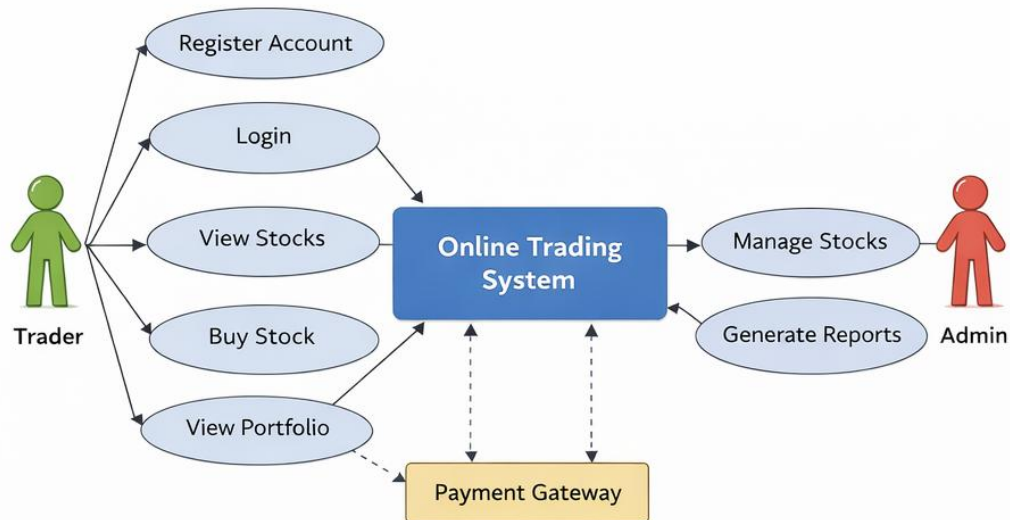
Use Cases :

- Register Account
- Login
- View Stocks
- Buy Stock
- Sell Stock
- Manage Portfolio
- Generate Reports

Use Case Scenario : Buy Stock

- Trader logs into system.
- Trader selects stock.
- System checks balance.
- Payment is processed.
- Portfolio updated.
- Confirmation displayed.

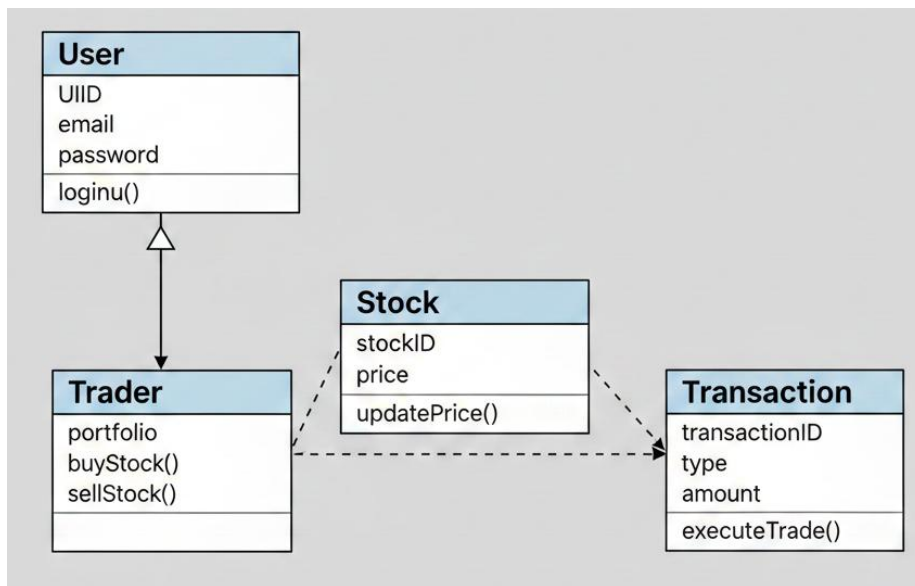
Use Case Diagram :



Task 3 : Class Diagram

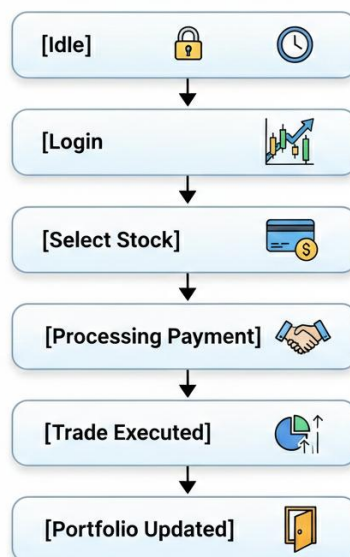
Classes Identified :

- User
- Trader
- Admin
- Stock
- Portfolio
- Transaction
- Payment



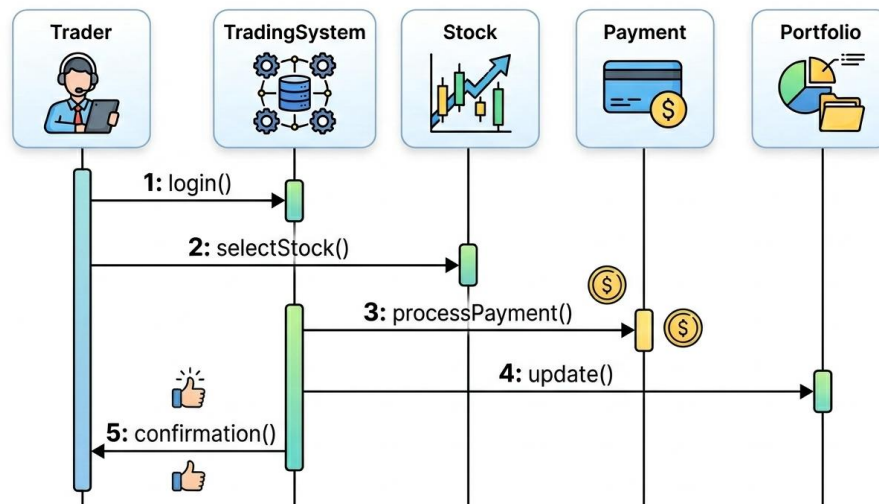
Task 4 : State Diagram

States of trading process :

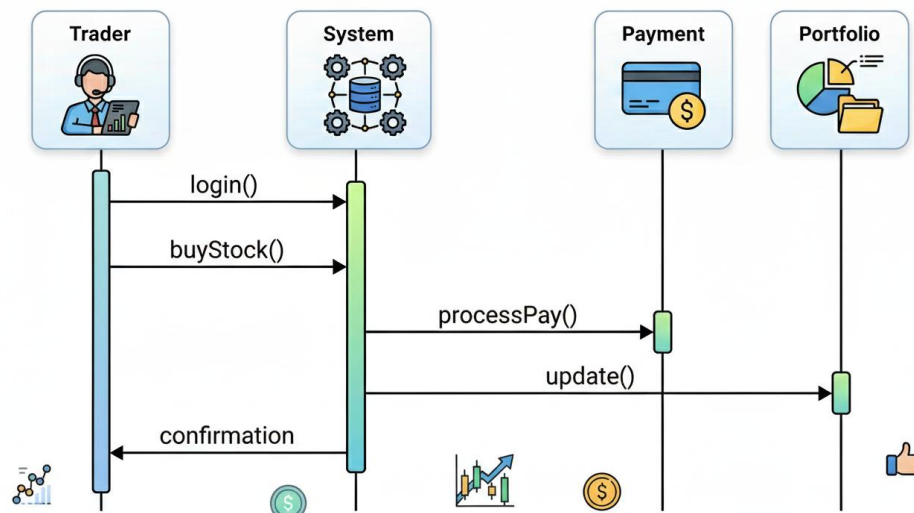


Task 5 : Collaboration Diagram

- object interaction

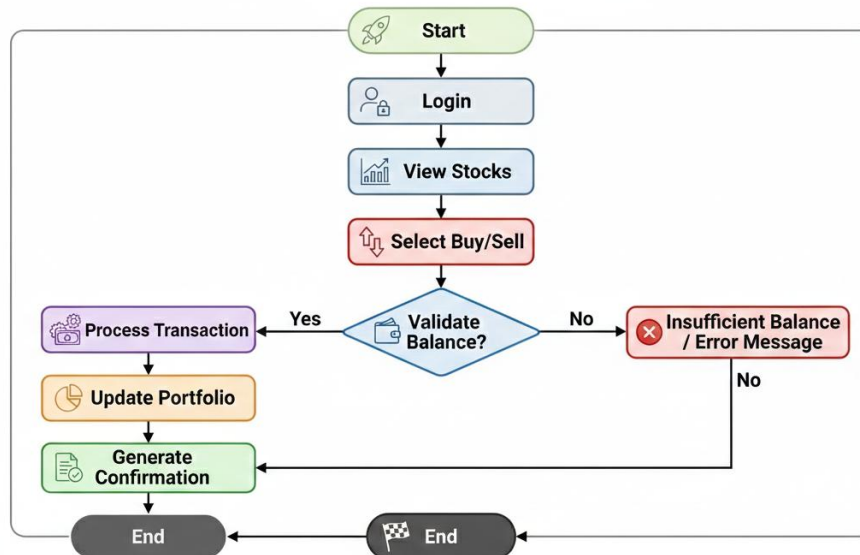


Task 6 : Sequence Diagram



Task 7 : Activity Diagram

STOCK TRADING WORKFLOW

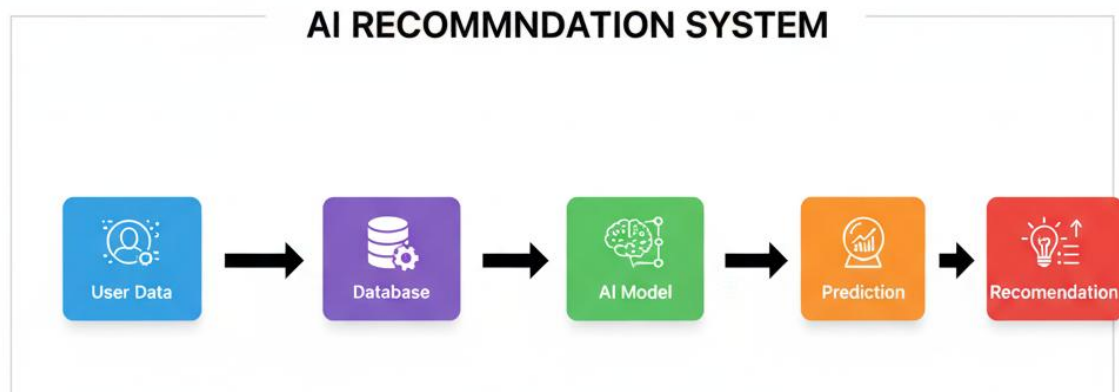


Task 8: AI Integration (Optional Enhancement)

AI Features in Online Trading System:

- Stock trend prediction
- Risk analysis
- Investment recommendation
- Fraud detection

AI RECOMMNDATION SYSTEM



Expected Outcomes :

- Complete UML design of Online Trading System.
- Proper object-oriented modeling.
- Clear representation of trading workflow.
- Documentation ready for implementation.

Practical 8

Aim :

Defining Coding Standards and Walkthrough.

Prerequisites :

- Before starting development, ensure the following requirements are available:

Programming Language :

- Java / Python / JavaScript (any preferred language)

Libraries / Frameworks :

- Backend: Django / Flask / Node.js / Spring Boot
- Frontend: HTML, CSS, JavaScript, React (optional)
- Database: MySQL / PostgreSQL / MongoDB
- API Integration: Stock Market APIs (for price updates)

Development Environment :

- IDE: VS Code / IntelliJ / PyCharm
- Browser for testing

Tools :

- Version Control: Git & GitHub
- Database Management Tool
- Postman (API Testing)

Coding Standards :

- To maintain readability, scalability, and maintainability, follow these standards:

General Guidelines :

- Use meaningful variable names (e.g., calculatePortfolioValue).
- Follow proper naming conventions.
- Write modular and reusable code.
- Add comments and documentation for each function.
- Maintain consistent indentation and formatting.
- Use Git commits with clear messages.

File Structure :

```
online_trading_system/  
|  
├── database/      # Database scripts  
├── backend/       # Server-side logic  
├── frontend/      # UI files  
├── models/        # Data models  
├── utils/         # Helper functions  
├── tests/         # Unit testing files  
├── app.py / server.js  # Main application  
├── requirements.txt  
└── README.md
```

Error Handling:

- Use try-catch / try-except blocks.
- Validate user inputs before processing.
- Handle payment and transaction failures properly.
- Maintain error logs for debugging.

Ex: try:

```
execute_trade(order)
```

except Exception as e:

```
print("Transaction Failed:", e)
```

Testing:

- Perform unit testing for modules.
- Conduct integration testing.
- Validate transaction accuracy.
- Test login authentication and security features.

Testing tools:

- unittest / pytest
- Selenium (UI testing)

System Architecture

The Online Trading System consists of the following modules:

1. User Management Module :

- User registration and login
- Profile management
- Authentication and authorization

2. Market Data Module :

- Fetch live stock prices
- Display charts and trends
- Update market information

3. Trading Module :

- Buy and sell stock
- Order placement

- Order history tracking

4. Portfolio Management Module :

- Track investments
- Calculate profit/loss
- Asset allocation overview

5. Payment & Wallet Module :

- Deposit funds
- Withdraw balance
- Transaction records

6. Reporting Module

- Generate trade reports
- Portfolio summaries
- Performance analytics

Step-by-Step Implementation

Step 1: Setup Project

- Create project directory.
- Initialize Git repository.
- Install dependencies.

Ex: git init

 pip install django

Step 2: User Registration & Login

- Create user database table.

- Implement authentication system.

```
Ex : def register_user(username, password):  
      save_user(username, password)
```

Step 3: Market Data Integration

- Fetch stock prices using API.

```
Ex: import requests  
  
data = requests.get("API_URL")  
  
prices = data.json()
```

Step 4: Trading Functionality

- Implement Buy/Sell operations.

```
Ex: def buy_stock(user, stock, quantity):  
      update_portfolio(user, stock, quantity)
```

Step 5: Portfolio Management

- Calculate total investment value.

```
Ex: def calculate_portfolio_value(stocks):  
      total = sum(stock.price * stock.qty for stock in stocks)  
      return total
```

Step 6: Reporting Module

- Generate charts and summaries.

```
Ex : import matplotlib.pyplot as plt  
      plt.plot(dates, portfolio_value)  
      plt.title("Portfolio Growth")  
      plt.show()
```

Testing and Validation :

- Verify correct order execution.
- Validate balance deduction.
- Check portfolio updates
- Perform security testing for login system.

Evaluation Metrics :

- Transaction accuracy
- Response time
- System reliability

Conclusion :

This practical demonstrated the design and implementation of an **Online Trading System** using software engineering principles. By following coding standards, modular architecture, and systematic testing, a scalable and secure trading platform can be developed that allows users to trade financial assets efficiently and safely.